

Computer Science 4753 – Fall 2025 – Assignment 2
Neural Network Fundamentals

This assignment will give you experience implementing and training a multi-layer (3 layer) fully connected, feed forward network consisting of sigmoidal neurons, trained with stochastic gradient descent and back propagation.

It is divided into two parts. **Part 1 (DUE: Monday, October 13th @ 7:00am)** involves the coding and training of a very tiny three-layer fully connected, feed forward network with stochastic gradient descent and back propagation. Once you have completed Part 1 of the assignment you will know that your core neural network engine and training algorithms function properly. **Part 2 of the assignment (DUE: Tuesday, October 21st @ 7:00 am)** will utilize the network of Part 1 to solve the real-world problem of recognizing handwritten digits (0-9).

Successful completion of Part 1 of the assignment is a pre-requisite for proceeding to Part 2.

Part 1 : Coding and training a three-layer, fully-connected, feed-forward network with “stochastic” gradient descent and back propagation

Part 1 Due: Monday, October 13th, 2025 @ 7:00am

Part 1 requires that you manually implement and verify the functionality of the core neural network and training algorithms in Java. To do this, you will write a Java program that implements and trains a very tiny network consisting of three layers: an input layer with four nodes, a hidden layer with 3 nodes, and an output layer with 2 nodes.

The values computed by your Java program must be identical to the values calculated by the neural network spreadsheet posted to Canvas. Essentially you will instrument your neural network engine to include diagnostic print statements that show the value of every variable that is displayed in the Excel spreadsheet (“Lecture 10 - Supplemental Material”, posted under the “Lectures” folder on our Canvas site).

As shown in the spreadsheet and mentioned above, this network consists of three layers with four nodes in the input layer, three nodes in the hidden layer, and two nodes in the output layer.

Normally, the initial weights and biases of a network are randomized before training begins. In order to ensure that the output of your program matches the spreadsheet, you will need to pre-load the weights and biases with the following pseudo-randomly assigned values. Please note that when you move to the full implementation of the larger three-layer network used in Part two of this assignment you should initialize your network with random numbers rather than hard coding initial weights and biases as we are doing here.

Weights Connecting Level 0 to Level 1			
-0.21	0.72	-0.25	1
-0.94	-0.41	-0.47	0.63
0.15	0.55	-0.49	-0.75

Biases
0.1
-0.36
-0.31

Weights Connecting Level 1 to Level 2		
0.76	0.48	-0.73
0.34	0.89	-0.23

Biases
0.16
-0.46

The learning rate “eta” will be set to 10.

The training data for this very tiny network will consist of four input/output pairs divided into two mini-batches of two examples each. In order that your program’s output match the spreadsheet, we will not randomize the mini-batches between epochs, as one should do when properly implementing stochastic gradient descent.

Please note that when you move to the full implementation of the larger three-layer network used in Part two of this assignment you should randomize the training data between mini-batches.

Here is the training data:

Mini-Batch # 1			
Input 1	Output 1	Input 2	Output 2
0		1	
1	0	0	1
0	1	1	0
1		0	

Mini-Batch # 2			
Input 3	Output 3	Input 4	Output 4
0		1	
0	0	1	1
1	1	0	0
1		0	

Run this network through six epochs of training with stochastic gradient descent and back propagation. During the sixth epoch your network should produce the following results on the four inputs.

Mini-Batch # 1			
Input 1	Output 1	Input 2	Output 2
0		1	
1	0.284737	0	0.686228
0	0.794477	1	0.227963
1		0	

Mini-Batch # 2			
Input 3	Output 3	Input 4	Output 4
0		1	
0	0.230277	1	0.692288
1	0.832609	0	0.20266
1		0	

What has this tiny network learned? While I wouldn't worry too much about that, I suppose we could summarize the training as having taught the network that while its first input is 0 and its last input is 1, its first output should be near 0 and its second output near 1; and also that when its first input is 1 and its last input is 0, its first output should be near 1 and its second output near 0. As you can see above, after six epochs the outputs are certainly leaning in this direction, training beyond six epochs should result in outputs even closer to the desired results specified in the input / output pairs.

Part 2 : MNIST Handwritten Digit Recognizer Neural Network in Java

Part 2 Due: Tuesday, October 21st, 2025 @ 7:00 am

Now that you have a working fully connected feed forward network and verified back propagation algorithms utilizing stochastic gradient descent, we can expand the core Java program to recognize the MNIST digit set.

The MNIST training and testing data, in CSV format, is available at <https://pjreddie.com/projects/mnist-in-csv/> and also on Canvas. The format of each line of data is: "label, pix-1-1, pix-1-2, pix1-3, ... , pix-28-28" where label is a digit 0-9 and pix-X-Y is a greyscale value from 0-255. NOTES: (1) The label will need to be converted to a "one hot vector" – a 10 by 1 array in order to work with our network; (2) For best results, scale your pixel inputs to be a fraction between 0 and 1 by dividing every pixel input by 255, storing the result as a double.

At a minimum, your program should support the following user-selectable operations:

[1] Train the network

In training mode, your program should iterate through the 60,000 item MNIST training data set. I used a learning rate of 3, a mini-batch size of 10, and 30 epochs; randomly initializing my weights and biases to the -1 to 1 range, and scaling my pixel inputs to the range of 0 – 1.

After each training epoch, your program should print out statistics showing: (1) for each of the ten digits [0-9], the number of correctly classified inputs over the total number of occurrences of that digit; (2) the overall

accuracy considering all ten digits. In other words, after each epoch, the output should look something like this:

```
0 = 4907/4932  1 = 5666/5678  2 = 4921/4968  3 = 5034/5101  4 = 4839/4859  5 = 4472/4506
6 = 4935/4951  7 = 5140/5175  8 = 4801/4842  9 = 4931/4988  Accuracy = 49646/50000 = 99.292%
```

(Note that I only used the first 50,000 items in the training set, which is why the denominator of my accuracy calculation is 50,000. There are actually 60,000 items in the training set. You should use all 60,000.)

[2] Load a pre-trained network

Your program should be able to load a weight set (previously generated) from a file.

[3] Display network accuracy on TRAINING data {only available after selecting [1] or [2] above}

Iterate over the 60,000 item MNIST training data set exactly once, using the current weight set, and output the statistics shown in [1] above.

[4] Display network accuracy on TESTING data {only available after selecting [1] or [2] above}

Iterate over the 10,000 item MNIST testing data set exactly once, using the current weight set, and output the statistics shown in [1] above.

[5] Run network on TESTING data showing images and labels. {only available after selecting [1] or [2]}

Show (while running the network on TESTING data) for each input image, a representation of the image itself, its correct classification, the network's classification, and an indication as to whether or not the network's classification was correct. (I used ASCII art for the image in order to stick with a command line program and provided the ability to return to the main menu after each image. See Figure 1.)

[6] Display the misclassified TESTING images {only available after selecting [1] or [2] above}

Similar to option [5] except only display the testing images that are misclassified by the network.

[7] Save the network state to file {only available after selecting [1] or [2] above}

Your program should be able to save the current weight set to a file.

[0] Exit

You should be able to exit the program.

```
Testing Case #446: Correct classification = 6 Network Output = 0 Incorrect.

      i
     o H
    . & ,
   H X .
  , H
 o H      . H H X X H X & & & H
 . H . ,   H X X & o o & & & H
 H & H X , k . . , , . X & & H
 , & X , k .                k & k
 k & i                      . & & k
 i & H                      o & o
 & & H                      o H o ,
 & & X                      , k & X .
 & & H                      . o & k i ,
 X & & & H . k X X X i .
 k o H & & & & o .
 . . X & & & i

Enter 1 to continue. All other values return to main menu.
```

Figure 1

Additional Requirements:

1. You must work independently and develop your own code. Sharing of code is ABSOLUTELY forbidden. Vibe coding is ABSOLUTELY forbidden. Lifting a solution from the internet or having an AI generate it will not help you either, as you must be able to explain how the code operates, and if you don't write it yourself you won't be able to do so. It is far better to turn in your own program, even if it only "partially" works, rather than some "perfect" program you did not write.
2. You **must** thoroughly document your program, including comments at the beginning of your main program with: your name, your student number, the date, the assignment number, and a brief description of what the program does. Comments should also be present throughout the program to explain what each part does. Lack of appropriate comments can cost you up to 20% of your overall program grade.
3. Your program must be written in Java and run in Eclipse or from the command line.

Submitting your assignment for grading:

For both Parts 1 and 2 of this assignment a copy of **your program should be emailed to me** (mike@LaTech.edu) **no later than 7am on the due date**. For Part 1 also include the output of a run of your program.

The program (and for Part 1 the output file) must be in a ZIP archive, as naked Java attachments are stripped by the mail server. The subject line of the email should be: **CSC 475 : Neural Network Program Part <1 or 2> : <your name>** where **<1 or 2>** is replaced with either a "1" or a "2" and **<your name>** is replaced with your name. So, for example, if Susan Calvin were enrolled in the class, her submission of Part 1 would use the subject line: **CSC 475 : Neural Network Program Part 1 : Susan Calvin**

For Part 2, in addition to submitting your program by the deadline, you will be required to **meet with me for approximately 15 minutes** to demo your program and explain how it works. I will place a signup sheet on my office door, or some other agreed upon location, to reserve a time block for your appointment. Come to the meeting prepared to demo your program. **Have a laptop with you, with your program pre-loaded in Eclipse.** Be sure you have a pre-trained weights file that can be loaded, so that you can demo your program's efficiency without us having to wait through a complete training cycle.

These meetings will determine your grade on Part 2 of the assignment. If you cannot explain to my satisfaction how your code works, or make simple changes to it, you will receive a grade of zero on the program (which being less than 50% will result in failure of the course).

Now, **don't panic!** If you wrote your own program, you will find your 15 minute meeting to be exceptionally easy to get through, and possibly even enjoyable.